applications from anywhere, making it ideal for remote and hybrid work environments. PEPs (edge nodes) enforce access, PE evaluates policies, and IdP verifies identity for distributed users. For example, Microsoft Azure inspects traffic continuously as it flows through edge nodes and Google Cloud enforces policies for every connection request from remote users.

Policy enforcement and dynamic access control: Access control is enforced through centralised policy engines that evaluate real-time signals. These dynamic policies ensure consistent and adaptive enforcement across all systems, improving security posture. The PE evaluates policies, the PA configures enforcement, and the PEP applies controls dynamically. Amazon Web Services evaluates policies on every API call or access request and Microsoft Azure re-applies policies whenever session risk or context changes.

Zero Trust across the modern stack

In the modern world, we can't expect everyone to be in the office, and the same goes for our apps. Many organisations now use a multi-cloud or cloud-to-cloud setup where data and services are spread across different providers. Instead of forcing every request to tunnel all the way back to the main headquarters, which just slows everything down, Zero Trust allows an application in one cloud to connect directly and securely to a data source in another. We achieve this by placing Policy Enforcement Points (PEPs) at the actual access points of each service, making the security as mobile as the data itself.

For SaaS and remote access, we have to protect 'subjects' (users or non-human entities) and assets that live outside our own office walls. Since these cloud-based resources are not inside a physical perimeter, we use identity-driven policies to manage access, making sure we know exactly who is requesting what, even if they are using a public network. We also assume that these local networks, like a home Wi-Fi or public hotspot, are hostile, so we insist on authenticating every connection and encrypting all traffic by default.

To make this manageable for developers, we move towards Policy as Code (PaC) and integrate security right into our CI/CD pipelines. This means we set up automated guardrails to check for things like correct identity attributes and proper encryption before anything even goes live. By treating security as a core part of the system design rather than an afterthought, we ensure that every new piece of code follows the 'Never trust, always verify' rule from day one.

Roadmap for developers: Step-by-step rollout

Stage 1: Discover and protect

The first step is to reach a baseline of competence by identifying and cataloguing all enterprise subjects, assets, and business processes. You must create a detailed inventory of physical and virtual devices, data classes, and the 'actors' (both human and service accounts) active on your network. Once you have mapped your critical business applications and their dependencies, the priority is to enable strong authentication, specifically phishing-resistant multi-factor authentication (MFA), to protect access to these resources.

Stage 2: Contain and control

After gaining visibility, you move into the 'contain and control' phase by formulating dynamic, risk-based access policies. This involves implementing 'conditional access', where the system evaluates the subject's identity and the device's security posture before granting a session. You should also introduce just-in-time (JIT) administration and expiring privileges to ensure that access is granted with the least privilege needed for a specific task and only for a limited duration. During this stage, it is wise to start a pilot microsegmentation project for a few 'crown jewel' or high-value assets (HVAs) to prove the concept without disrupting the whole organisation.

Stage 3: Microsegment and automate

In the final stage, you enforce full segmentation for all critical apps and services, ensuring they are not even discoverable without going through a Policy Enforcement Point (PEP). At this point, your Policy Engine (PE) should be in a steady operational phase, using centralised telemetry from SIEM and SOAR systems to automate risk responses. By implementing 'Policy as Code' and automated guardrails, you ensure that security is enforced consistently across the entire enterprise, allowing your team to focus on monitoring and refining policies based on real-time feedback.

Success metrics and pitfalls

Success metrics (KPIs):

- Track your Mean Time to Detect (MTTD) and Mean Time to Respond (MTTR) using machine learning and ZTA logs to see if the architecture is helping you catch and stop threats faster.
- Measure the percentage of your resources and subjects, especially those with sensitive access that are protected by multi-factor authentication (MFA).
- Monitor the decrease in permanent, 'blanket' admin accounts in favour of dynamic, per-session access governed by a Policy Engine.
- Use network and system activity logs to identify how many unauthorised attempts to move between network segments were successfully stopped by your Policy Enforcement Points (PEPs).

Continued to page....90